

CP372 – Computer Networks

Assignment 1 – TCP Client-Server Application

Protocol Documentation

Instructor: Dr. Tao Xu

Averi Wylie

Yakup Bastug

Efe Karagoz

Faruk Genc

Dylan Baetz

System Overview:

In this work, a client-server TCP application is developed using Socket API from Python. The system enables the client to establish connection with the server, authenticate by username, message in text format, send files, and finally terminate the connection.

The application employs the client-server architectural design in which:

- 1) The server waits for TCP connections.
- 2) A single client connects to the server at a time.
- 3) The client uses commands in text format under an application protocol.
- 4) The server processes the requests and provides responses.
- 5) Files are transferred reliably using TCP protocol and saved on the server.

The application enables the following functionalities:

COMMAND	PURPOSE
LOGIN	Authenticate user
MSG	Send text messages
FILE	Transfer files
QUIT	Disconnect from the server

Protocol Specification:

This application works with a text-based protocol that is implemented above TCP. Every request is sent as a line of text, UTF-8 encoded and ended with “\n”. General structure; COMMAND arguments, responses from the server may take the following form:

OK<message>

ERR<message>

Where: OK indicates a successful processing and ERR indicates an error occurred.

Synchronization (part of protocol specification)

Connection establishment requires the server to be running before any client attempts to connect. The client initiates the TCP connection; the server accepts it. Once connected, the session proceeds in a strict command-response pattern: the client sends one command, waits for the server's response, then sends the next. Neither side sends unsolicited messages. Authentication is a synchronization gate: the server will not process MSG or FILE until LOGIN has succeeded, rejecting those commands with ERR until the gate is cleared. The file transfer sub-sequence (FILE → OK READY → SIZE → OK SEND → raw bytes → OK FILE succeeded) is a blocking six-step handshake within the authenticated state; no other commands are valid until it completes. Session termination is initiated by the client via QUIT; the server responds OK QUIT succeeded and closes the socket. If the client disconnects without QUIT, the server detects an empty recv(), logs the event, and closes cleanly without crashing. There are no application-level timeouts; the protocol relies on TCP's built-in reliability.

Command case sensitivity (part of protocol specification)

Commands are case insensitive. The client and server both normalize incoming commands to uppercase before parsing, so login, LOGIN, and Login are all treated identically. Arguments such as usernames are case sensitive and are passed through as provided. The canonical and recommended form for all commands is uppercase as shown throughout this document.

Command Descriptions

LOGIN - LOGIN <username>

The LOGIN <username> works to authenticate the session. The server checks for a matching pair of the entered username in the users.txt. If users.txt doesn't exist, the server starts with a fresh empty set

and rejects all logins instead of crashing. The login request needs to succeed before other commands are initialized.

MSG - MSG <text>

Sends a message in the UTF-8 format. The server records the time and the send status and prints it to the console. An empty argument is met with an ERR. A success case is printed to the console in the format ([MSG] [YYYY-MM-DD HH:MM:SS] username: text) along with an **OK MSG succeeded** printout.

FILE - FILE <filepath>

Initiates a file transfer to the server. The client verifies the file exists and is compatible before it strips down the path just to the base filename using **os.path.basename()** on both the client and server side as a path guard. The transfer consists of a 6-step handshake, with back and fourth client and server responses. For the detailed handshake process, see File Transfer Workflow5.

QUIT

Terminating the session can be done by both sides. The standard is that the client sends QUIT which the server replies with an OK QUIT message and shuts down the socket. If the client disappears without sending QUIT (in a crash scenario), the server sees an empty read from `recv()`, logs it as an unexpected disconnect message and quits cleanly.

Message Structure:

This application implements communication via a text protocol which is an extension to TCP.

Communication between the client and the server is implemented through commands from the client side and responses from the server side. A command is sent using UTF-8 encoding as a text message ending in a newline (`\n`) character. The server receives the command, parses it, and sends back a response depending on whether the operation succeeded or not.

- CLIENT TO SERVER MESSAGES -

Client messages follow the general format - COMMAND <arguments> where COMMAND identifies the requested operation and arguments containing any required parameters. Commands are case insensitive, although uppercase is recommended. Each Command must be sent on a separate line.

-SERVER TO CLIENT MESSAGES-

All server responses begin with a status code followed by a descriptive message. General format - STATUS message. EX: Successful login - OK Welcome, Yakup.

-PROTOCOL VALIDATION RULES-

The server validates all incoming messages before processing them. Validation rules include;

- 1) User must authenticate before using MSG or FILE commands
- 2) FILE commands must include a valid filename
- 3) File sizes must be non negative integers
- 4) Empty commands are rejected
- 5) Unknown commands generate an error response
- 6) Connection termination must use the QUIT command whenever possible

File Transfer Workflow

File transfer is a six-step handshake between the server and client/

STEP 1	Client -> Server	FILE <filename>\n	Client sends the stripped base filename.
STEP 2	Server -> Client	OK READY\n	Server understands the transfer and gets ready to receive size information.
STEP 3	Client -> Server	SIZE <bytes>\n	Client sends the exact file size in bytes as a decimal integer.
STEP 4	Server -> Client	OK SEND\n	Server confirms the size and tells the client to begin sending data
STEP 5	Client -> Server	<raw binary bytes>	Client starts streaming exact SIZE bytes

			with no encoding, no newline. Sent in chunks of up to BUFFER_SIZE (4096 BYTES)
STEP 6	Server -> Client	OK FILE succeeded\n	Server confirms it received and saved exactly SIZE bytes to server_files/.

Path Traversal Prevention

Both client and server call the command `os.path.basename()` on the filename before use. The client strips any directory prefix the user provides (e.g. `../../etc/passwd` → `passwd`), and the server does it again on the received name before adding the path to `server_files/`. This prevents malicious client attempt from writing files outside of the intended directory.

ERROR HANDLING

All errors return an `ERR <reason>` response to the client. The server never crashes and catches exceptions that aren't protocol errors.

LOGIN with unknown username	ERR Unknown user '<x>'. Access denied.
LOGIN with no username argument	ERR LOGIN requires a username. Usage: LOGIN <username>
MSG before LOGIN	ERR You must LOGIN before sending messages.
MSG with empty text	ERR MSG requires text. Usage: MSG <text>
FILE before LOGIN	ERR You must LOGIN before transferring files.
FILE with no filename argument	ERR FILE requires a filename. Usage: FILE <filename>
FILE with invalid/empty basename	ERR Invalid filename.

SIZE line not received after FILE	ERR Expected SIZE <bytes> after FILE command.
Non-integer value in SIZE line	ERR Invalid file size.
Negative value in SIZE line	ERR File size cannot be negative.
Connection drops during file transfer	ERR File transfer interrupted. (connection closed)
Unknown command	ERR Unknown command '<X>'. Valid commands: LOGIN, MSG, FILE, QUIT
Empty line received	ERR Empty command received.
Client disconnects without QUIT	(no response – server detects empty recv and closes cleanly)
users.txt missing at startup	(server starts with empty user set; warning printed to console)
File not found on client side	[ERROR] File not found: '<path>' (never sent to server)

Design Choices Organized

Raw bytes for file transfer Text and binary files are both sequences of bytes. Decoding the payload to text and re-encoding it would add nothing and risks corruption from encoding mismatches or newline translation. One raw-byte transfer path preserves the exact contents of any file type without special casing.

Single-threaded architecture A single thread is sufficient because the server is always blocked in one place at a time: either in `accept()` waiting for a connection, or in `recv()` waiting for a command. Multithreading would only be needed to serve multiple clients simultaneously, which is outside the scope of this assignment. No CPU is wasted; blocking socket calls yield to the OS while waiting.

Line-by-line protocol with 1-byte reads Commands and responses are plain UTF-8 lines terminated by `\n`. Reading one byte at a time until the newline delimiter is found prevents over-reading into the next command or into binary file payload bytes. This is particularly critical at the boundary between control lines and raw file data. After the server reads the `SIZE` line and responds `OK SEND`, the very next bytes on the socket are the raw binary file contents. If `recv_line()` used a buffer larger than one byte, it would pull those file bytes into the line buffer while looking for a `\n`, consuming bytes that belong to the payload. The server would then read fewer bytes than `SIZE` from the socket, the integrity check would fail, and the stored file would be silently corrupted or truncated. By reading exactly one byte per call, `recv_line()` stops precisely at the `\n` and leaves the socket position at the first byte of the file payload, ready for the raw byte loop. The same issue applies between consecutive commands: a buffered read on a `MSG` response could consume the first bytes of a subsequent `FILE` command. The 1-byte approach guarantees clean boundaries throughout the entire session regardless of what follows.

File overwrite behavior Files received by the server are written directly to `server_files/` using the base filename. If a file with the same name already exists in that directory it is silently overwritten. There is no versioning, renaming, or conflict detection. This is intentional for simplicity: the assignment requires reliable transfer and storage, not file management. Users should be aware that sending a file with a duplicate name replaces the previous version permanently.

Double-sided path sanitization Both the client and server call `os.path.basename()` on the filename before use. The client strips directory prefixes before sending; the server strips again on receipt before writing to `server_files/`. This ensures the server never trusts client input alone when constructing file paths, preventing directory traversal attacks regardless of client behavior.

Client-side pre-validation The client validates certain conditions locally before sending anything to the server. File existence and path validity are checked on the client side first, and if the check fails the command is aborted without contacting the server. This means some error messages like `[ERROR] File not found` come from the client, not the server, and will appear even before `LOGIN`

(i.e. if a file doesn't exist, the error message will not be about login because the command wasn't valid to see the server side of the architecture). This is intentional: there is no point sending a FILE command for a file that does not exist, and catching it early avoids a wasted round trip and a partial handshake the server would have to clean up. It's a sacrifice of usability for efficiency.

Interrupt and crash handling Three disconnection paths exist beyond the standard QUIT command. First, on the client side, Ctrl+C is caught by a KeyboardInterrupt handler which sends QUIT to the server before closing the socket, making it a graceful disconnect rather than an abrupt one. This is a deliberate shortcut for users who prefer keyboard interrupts over typing QUIT. Second, if a client process is killed or crashes without sending QUIT, the server detects an empty `recv()` result, logs the unexpected disconnect, closes the connection cleanly, and loops back to `accept()` to wait for the next client. Third, Ctrl+C on the server side is caught by its own KeyboardInterrupt handler which breaks the accept loop, closes the server socket cleanly, and prints a shutdown message rather than crashing. In all cases the server never goes down as a result of abrupt termination on either side.